



Uso de JavaScript (III)

Llamadas AJAX mediante REST y JSON

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

**Carlos Arévalo, Daniel Ayala, José Calderón,
Margarita Cruz, Inma Hernández, David Ruiz**

UNIVERSIDAD DE SEVILLA

Objetivos



Contenidos

REST

Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

Ejemplos

Módulos de comunicación con API



Restricciones arquitectónicas de REST

Arquitectura cliente-servidor

Sin estado

Cacheable

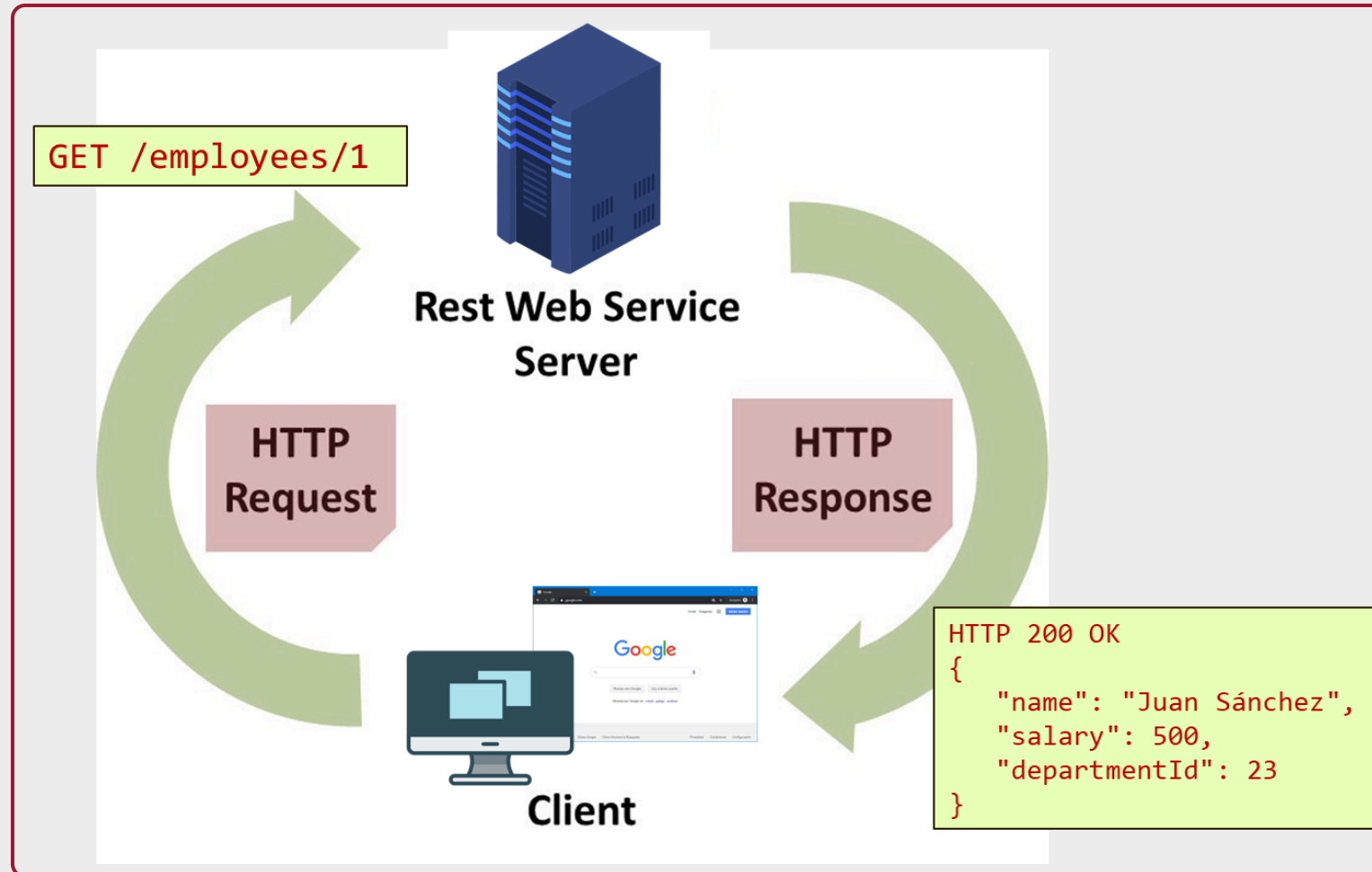
Modelo en capas

Interfaz uniforme

- Uso de URIs para identificar recursos
- HATEOAS
- Métodos HTTP estándar: GET , POST , PUT , DELETE , PATCH



Modelo de comunicación REST



Contenidos

REST

Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

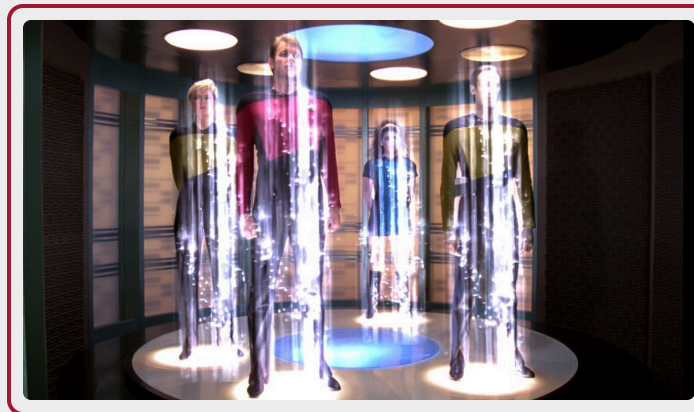
Ejemplos

Módulos de comunicación con API

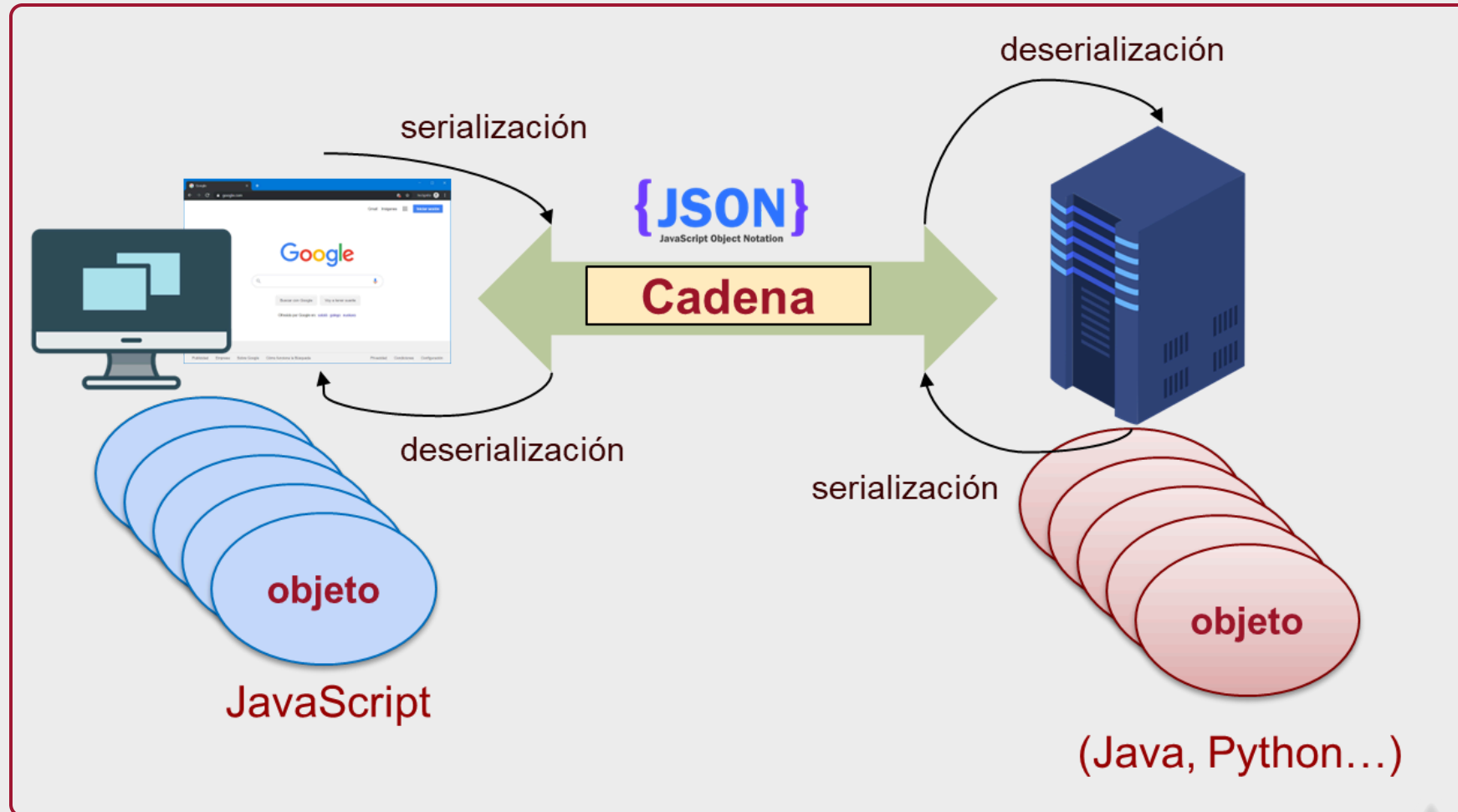
Serialización

es el proceso de codificación de objetos o estructuras de datos a un formato que permita ser almacenados (por ejemplo, en un fichero o un buffer de memoria) o transmitidos (por ejemplo, a través de una conexión de red), y posteriormente reconstruidos (posiblemente, en un sistema informático diferente).

Al proceso contrario (la reconstrucción de los objetos) se le denomina “deserialización”.



Serialización para transmisión de datos



JSON

Formato de intercambio de datos abierto

- También se usa, por ejemplo, como formato de ficheros de configuración

Basado en la notación para Objects y Arrays de JS (**JavaScript Object Notation**)

Usa una sintaxis muy comprensible para humanos

Los objetos se representan mediante un conjunto de pares atributo-valor

También permite representar arrays de objetos o valores

Independiente del lenguaje de programación

Estándar ECMA desde 2013 (ECMA-404) e ISO desde 2017 (STD-90, ISO/IEC 21778:2017)



Douglas Crockford

Tipos de datos

`Number` : Números decimales con signo, incluyendo notación exponencial

`String` : Secuencia de caracteres Unicode entre signos de comillas dobles. Los caracteres especiales (tildes, etc.) se pueden escapar con `\`

`null` : Valores vacíos. Sirve para representar también los 'undefined' de JavaScript

`Array` : Elementos separados por comas y encerrados entre corchetes `[]`. Pueden ser de cualquier tipo (incluyendo objetos y otros arrays)

`Object` : Colección no ordenada de pares nombre: valor, encerrados entre llaves `{ }` y separados por comas. Arrays asociativos (el nombre es la clave)

`Boolean` : `true` / `false`

Sintaxis

```
{
  "employees": [
    {
      "id": 1,
      "name": "Juan Ruiz",
      "salary": 1500
    },
    {
      "id": 2,
      "name": "Ana Ferrer",
      "salary": 1700
    }
  ]
}
```

Especificación: <https://www.json.org/json-en.html>

Otros elementos de la sintaxis

Se permite el uso de espacios en blanco

- Si están alrededor de los elementos, se ignora
- Si están dentro de una cadena, se consideran

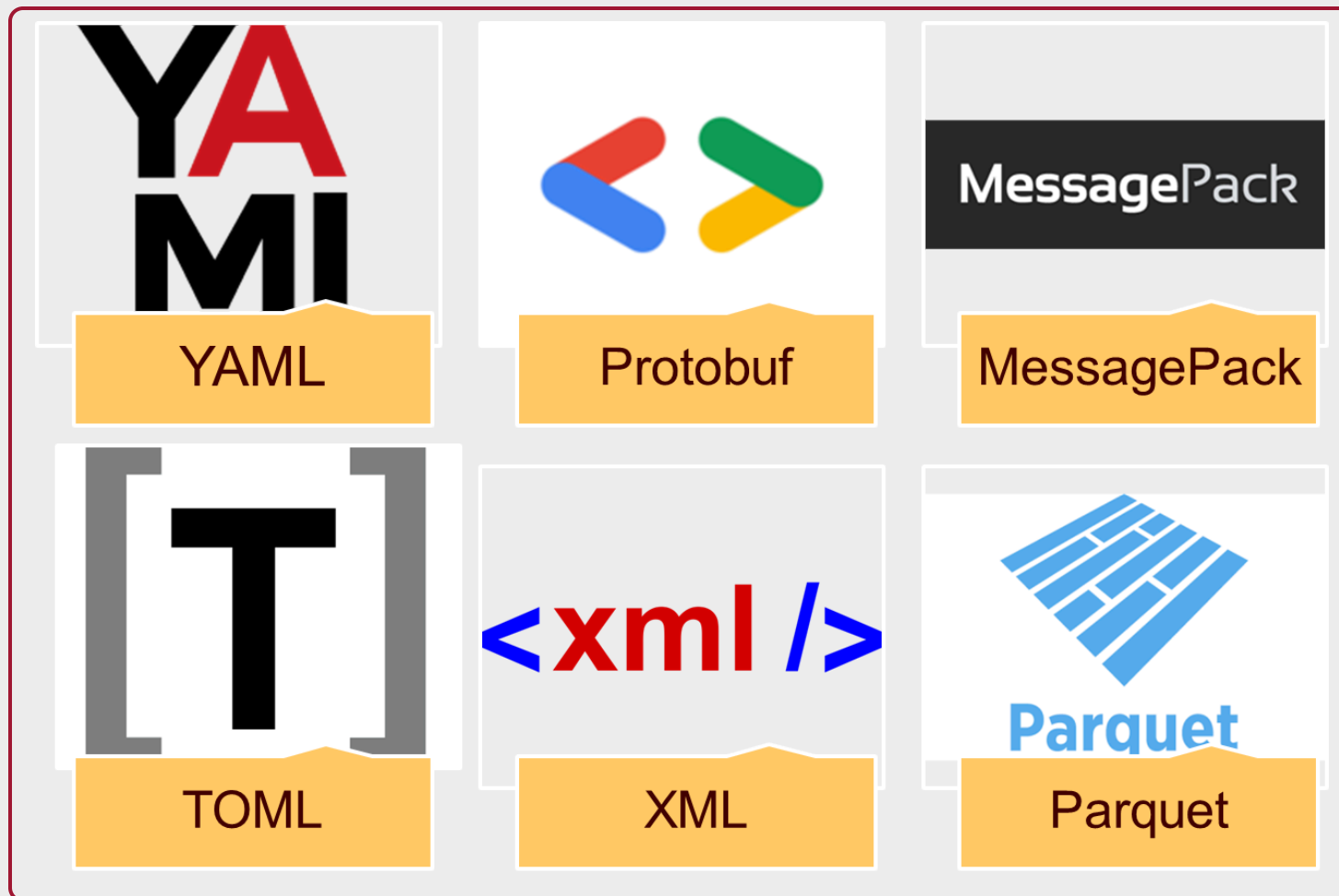
No se permiten los comentarios

Tipo MIME: application/json

La indentación no es necesaria, pero permite una mejor lectura

- Para mejorar la eficiencia en la transmisión, se suele eliminar la indentación al enviar JSON por la red
- Se puede volver a aplicar indentación usando herramientas(p.ej. <https://codebeautify.org/jsonviewer>)

Alternativas



Contenidos

REST

Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

Ejemplos

Módulos de comunicación con API

Comunicación síncrona

Cuando hay comunicación síncrona entre dos entes A y B, si A envía un mensaje a B, deja de actuar hasta que B responde

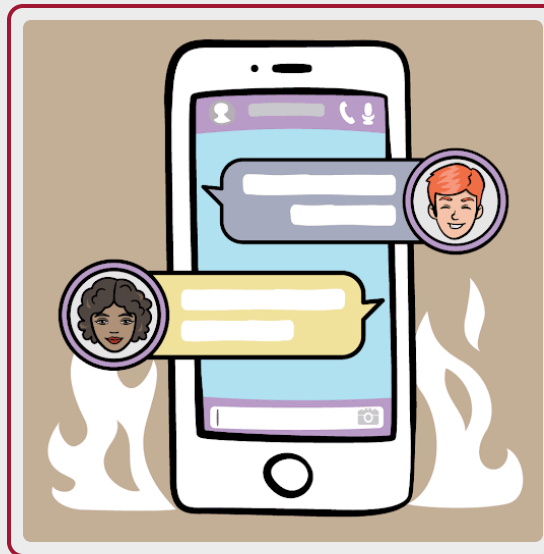
- También llamada “bloqueante”, porque A se bloquea esperando



Comunicación asíncrona

Cuando hay comunicación asíncrona entre dos entes A y B, si A envía un mensaje a B, puede seguir actuando (incluso enviando otros mensajes a B o otros entes). B responderá cuando pueda y enviará un mensaje de vuelta a A

- También llamada “no bloqueante”



Contenidos

REST

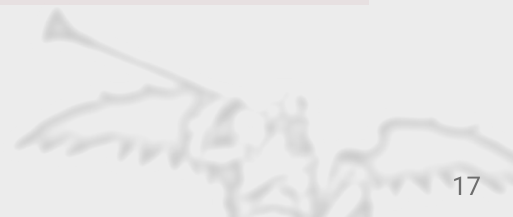
Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

Ejemplos

Módulos de comunicación con API



AJAX

AJAX (**A**synchronous **J**avaScript **A**nd **X**ML): Término acuñado en 2005 para describir una nueva forma de abordar las peticiones HTTP al servidor para hacer actualizaciones parciales de la página en cliente, sin tener que hacer una recarga completa.

Inicialmente, se usaba XML como formato de comunicación, pero hoy en día lo habitual es usar JSON

El núcleo de AJAX es la API XMLHttpRequest (XHR) de JS

- Documentación: <https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest>

Algunas implementaciones

- Nativas: XHR, fetch
- Librerías externas: jQuery, axios

XMLHttpRequest (XHR)

XHR puede usarse para hacer peticiones síncronas o asíncronas

- Parámetro opcional “async” del método open()
- Por defecto, asíncronas
- Se recomienda usar peticiones asíncronas, que ofrecen una experiencia más fluida al usuario

Se basa en el uso de funciones callback

- Se asocian funciones que se ejecutarán cuando ocurran ciertos eventos sobre la petición, como que se haya respondido o que haya ocurrido un error.

Ventajas

- Soporte nativo
- Permite gran control a bajo nivel sobre la petición

Desventajas

- La interfaz a bajo nivel hace que sea complejo de utilizar



Fetch

API nativa de JavaScript basada en promesas

- Documentación: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

La respuesta a una petición hecha con fetch es una promesa

- Dado que la comunicación es asíncrona, no se pueden devolver los datos inmediatamente, solo la promesa de que cuando lleguen los datos, se ejecutará un cierto fragmento de código

Cuando se resuelve la promesa, se puede consultar el objeto `Response` y sus propiedades: `Status`, `statusText`, `Headers`, ...

Ventajas

- Nativo, interfaz más a alto nivel que XHR

Desventajas

- La compatibilidad con navegadores no es del 100% aún
- La lógica para obtener los datos de una respuesta requiere encadenar como mínimo dos promesas y es menos limpia que otras alternativas

jQuery

La librería jQuery incluye (entre otras muchas cosas) diversos métodos para realizar peticiones AJAX

- Documentación: <https://api.jquery.com/category/ajax/>

Basada en XMLHttpRequest (XHR)

Ventajas

- Respecto a XHR: Facilita la escritura de código para realizar peticiones AJAX
- Respecto a fetch: Al ser parte de jQuery, es cross-browser

Desventajas

- Librería externa
- Enviar formularios mediante objetos FormData nativos requiere una configuración poco intuitiva
- La librería jQuery está cayendo cada vez más en desuso al tener los navegadores una API cada vez más estándar

Axios

Librería basada en XMLHttpRequest (la usa internamente), pero usa funciones asíncronas en vez de callbacks.

Ventajas

- Intercepta `request` y `response` : permite examinar y modificar las peticiones y respuestas HTTP (por ejemplo, para login o autenticación)
- Permite cancelar requests
- Múltiples librerías de terceros para ampliar la funcionalidad
- Compatibilidad con muchos navegadores, incluso antiguos
- Simplifica la lógica de gestión de respuestas y errores
- Muy popular actualmente, disponible también para su uso en backend con NodeJS

Desventajas

- Librería externa

Contenidos

REST

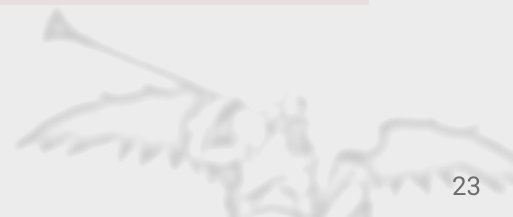
Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

Ejemplos

Módulos de comunicación con API



Peticiones GET

Necesitamos importar axios:

```
<script src="https://unpkg.com/axios/dist/axios.min.js"></script>
```

Esquema base:

```
await axios.get/post/put/delete(url, requestOptions)
```

Petición GET (I)

Ejemplo: consultar todas las fotos

```
try {  
  let response = await axios.get("/api/v1/photos");  
  let photos = response.data;  
  console.log(photos);  
} catch (err) {  
  console.error(err);  
}
```

```
▲ [Object Array]: [Object, Object, Object, Object, Object, Object, Object, Objec  
t, Object, Object, Object]  
  ▲ 0: Object  
    date: "Mon, 23 Apr 2012 18:25:43 GMT"  
    description: "A typical Spanish tortilla. With onion, of course."  
    photoId: 1  
    title: "Tortilla"  
    url: "https://cocina-familiar.b-cdn.net/recetas/thumb/tortilla-de-patata-con-cebolla.jpg"  
    userId: 1  
    visibility: "Public"  
    ▸ __proto__: Object  
  ▸ 1: Object  
  ▸ 2: Object  
  ▸ 3: Object  
  ▸ 4: Object  
  ▸ 5: Object  
  ▸ 6: Object  
  ▸ 7: Object  
  ▸ 8: Object  
  ▸ 9: Object  
    length: 10
```

Petición GET (II)

El objeto `response`

```
try {  
  let response = await axios.get("/api/v1/photos");  
  let photos = response.data;  
  console.log(photos);  
} catch (err) {  
  console.error(err);  
}
```

Todas las peticiones con axios devuelven un `response`, cuyos principales atributos son:

- `data`: Datos devueltos por el servidor, ya deserializados
- `status`: Código HTTP devuelto
- `statusText`: Mensaje textual del código HTTP devuelto
- `headers`: Cabeceras de la respuesta HTTP del servidor
- `request`: Referencia a la petición axios asociada a esta respuesta

Petición GET (III)

Ejemplo: consultar una URL que no existe

```
try {  
  let response = await axios.get("/api/v1/9999999999");  
  let photos = response.data;  
  console.log(photos);  
} catch (err) {  
  console.error(err);  
}
```

```
❗ HTTP404: NO ENCONTRADO: el servidor no encontró nada que coincidiera con  
el URI (identificador uniforme de recursos) solicitado.  
(XHR)GET - http://localhost:8080/api/v1/adsfasgfasf  
  
❗ [object Error]: {config: Object, description: "Request failed with stat  
us code 404", isAxiosError: true, message: "Request failed with status  
code 404", request: XMLHttpRequest...}  
  ▶ config: Object  
    description: "Request failed with status code 404"  
    isAxiosError: true  
    message: "Request failed with status code 404"  
  ▶ request: XMLHttpRequest  
  ▶ response: Object  
    stack: "Error: Request failed with status code 404 at e.exports (htt  
p://localhost:8080/js/libs/axios.min.js:2:11274) at e.exports (htt  
p://localhost:8080/js/libs/axios.min.js:2:11098) at l.onreadystatechange  
ange (http://localhost:8080/js/libs/axios.min.js:2:9711)"  
  ▶ toJSON: function () { return { message: this.message, name: this.nam  
e, description: this.description, number: this.number, fileName: thi  
s.fileName, lineNumber: this.lineNumber, columnNumber: this.columnNu  
mber, stack: this.stack, config: this.config, code: this.code } }  
  ▶ __proto__: Error
```

Petición GET (IV)

El objeto error

```
try {  
  let response = await axios.get("/api/v1/9999999999");  
  let photos = response.data;  
  console.log(photos);  
} catch (err) {  
  console.error(err);  
}
```

Cualquier petición que devuelva un código HTTP de error (4XX, 5XX) eleva una excepción con un objeto Error, cuyos principales atributos son:

- `description` : Descripción textual del error
- `request` : Referencia a la petición axios que provoca el error
- `response` : Respuesta del servidor, con los mismos atributos que una respuesta normal (`data` , `status` ...)

Petición GET (V)

En cualquier petición axios se puede pasar un parámetro adicional para configurar la petición (cabeceras HTTP, timeout...)

```
let options = {
  headers: {
    Token: "ABCDE123",
  },
  timeout: 5000, // milliseconds
};

try {
  let response = await axios.get("/api/v1/photos", options);
  let photos = response.data;
  console.log(photos);
} catch (err) {
  console.error(err);
}
```

Peticiones POST/PUT

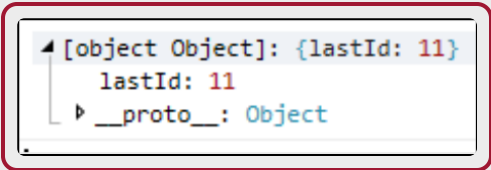
Esquema base:

```
await axios.post/put(url, body, requestOptions)
```


Petición POST/PUT (I)

En las peticiones POST y PUT se puede pasar un objeto JS, que se serializará a JSON y se enviará como cuerpo de la petición

```
let photo = {  
  url: "/images/example.jpg",  
  date: "2020-04-20 13:37:00",  
  title: "New photo",  
  description: "A new photo",  
  visibility: "Public",  
  userId: 2  
};  
  
try {  
  let response = await axios.post("/api/v1/photos", photo);  
  console.log(response.data);  
} catch (err) {  
  console.log(err);  
}
```



```
[object Object]: {lastId: 11}  
  lastId: 11  
  __proto__: Object
```

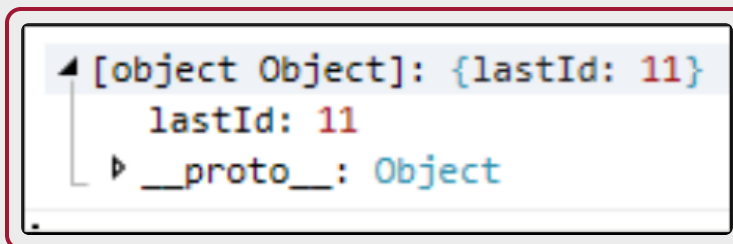
Petición POST/PUT (II)

Si los datos provienen de un formulario, se puede enviar un objeto FormData construido a partir del `<form>` para evitar tener que construir el cuerpo de la petición manualmente:

```
let form = document.getElementById("photo-form");
let formData = new FormData(form);

try {
  let response = await axios.post("/api/v1/photos", formData);
  console.log(response.data);
} catch (err) {
  console.log(err);
}
```

Las peticiones PUT son equivalentes usando el método `axios.put()`

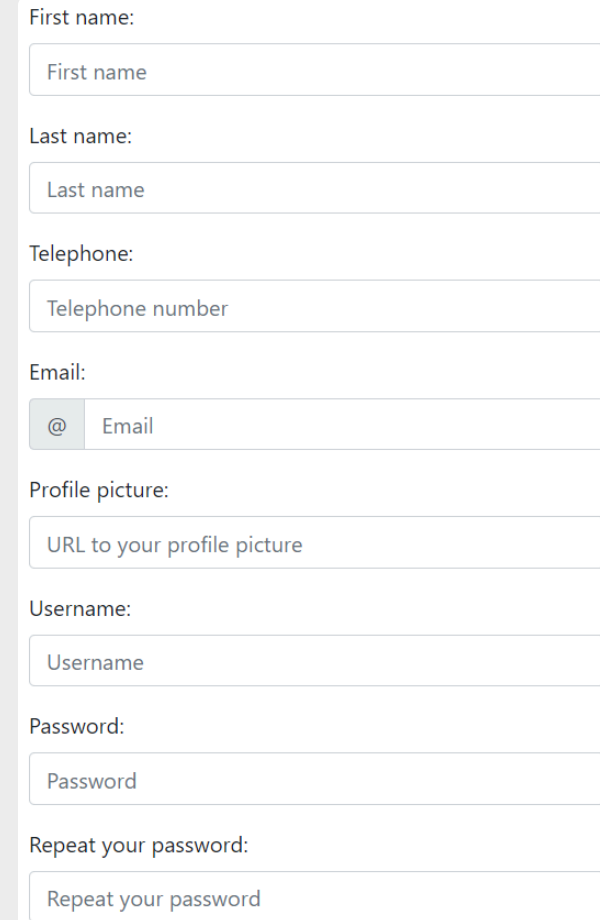


Ejemplo POST: Registro de usuario

```
<form id="register-form">
```

```
let form = document.getElementById("register-form");
let formData = new FormData(form);

try {
  let response = await axios.post("/api/v1/register",
    formData);
  console.log(response.data);
} catch (err) {
  console.log(err);
}
```



A user registration form with the following fields and labels:

- First name:
- Last name:
- Telephone:
- Email:
- Profile picture:
- Username:
- Password:
- Repeat your password:

Ejemplo POST: Registro de usuario

Se devuelve el usuario creado y el token de sesión



```

[object Object]: {sessionToken: ".eJw9jkFuwyAQRa9SoSyjFGzA4FWX6SbKxgcY48F
BwQYBThRFvXuxKnV28570_3-TLWP6nkjPj8S6lMsFFiQ9uaYNRyBH4uGfXTZ8hI8hb5BcqKqg
x3gL6-4k1YIKTnnluIDz1RXM5WsG7zG9TiYsVe1t61_ais-hfhVGyPkZUh1B4nifbNPnGzRC9
kzQegcnzHnqzteDxhGZ0oorTaltQbZWqM5YK3nbAaDRk2SWMoWyE1Y0igNQbQQzVLWG69oFDy
iQhrTvcwvMmD9jCtZ5PMV1Jj-_Oi9VDA.X0TPyQ.Z7V2DYK8bL-2ne49kDcWJ87pZHU", use
r: Object}
  sessionToken: ".eJw9jkFuwyAQRa9SoSyjFGzA4FWX6SbKxgcY48FBwQYBThRFvXuxKn
V28570_3-TLWP6nkjPj8S6lMsFFiQ9uaYNRyBH4uGfXTZ8hI8hb5BcqKqgx3gL6-4k1YIK
TnnluIDz1RXM5WsG7zG9TiYsVe1t61_ais-hfhVGyPkZUh1B4nifbNPnGzRC9kzQegcnzH
nqzteDxhGZ0oorTaltQbZWqM5YK3nbAaDRk2SWMoWyE1Y0igNQbQQzVLWG69oFDyiQhrTv
cwvMmD9jCtZ5PMV1Jj-_Oi9VDA.X0TPyQ.Z7V2DYK8bL-2ne49kDcWJ87pZHU"
  user: Object
    avatarUrl: "images/profile.png"
    email: "test@gallery.com"
    firstName: "Prueba"
    lastName: "Nuevo Usuario"
    telephone: "606505404"
    userId: 4
    username: "newUser"
    __proto__: Object
  __proto__: Object

```

Peticiones DELETE

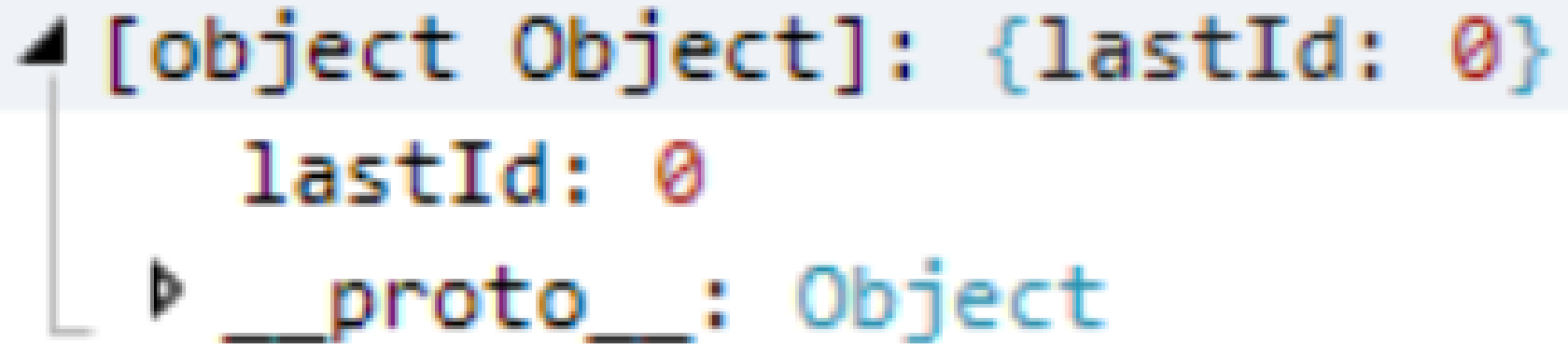
Esquema base:

```
await axios.delete(url, requestOptions)
```

Petición DELETE (I)

Sólo es necesario indicar la URL:

```
try {  
  let response = await axios.delete("/api/v1/photos/3");  
  console.log(response.data);  
} catch (err) {  
  console.log(err);  
}
```



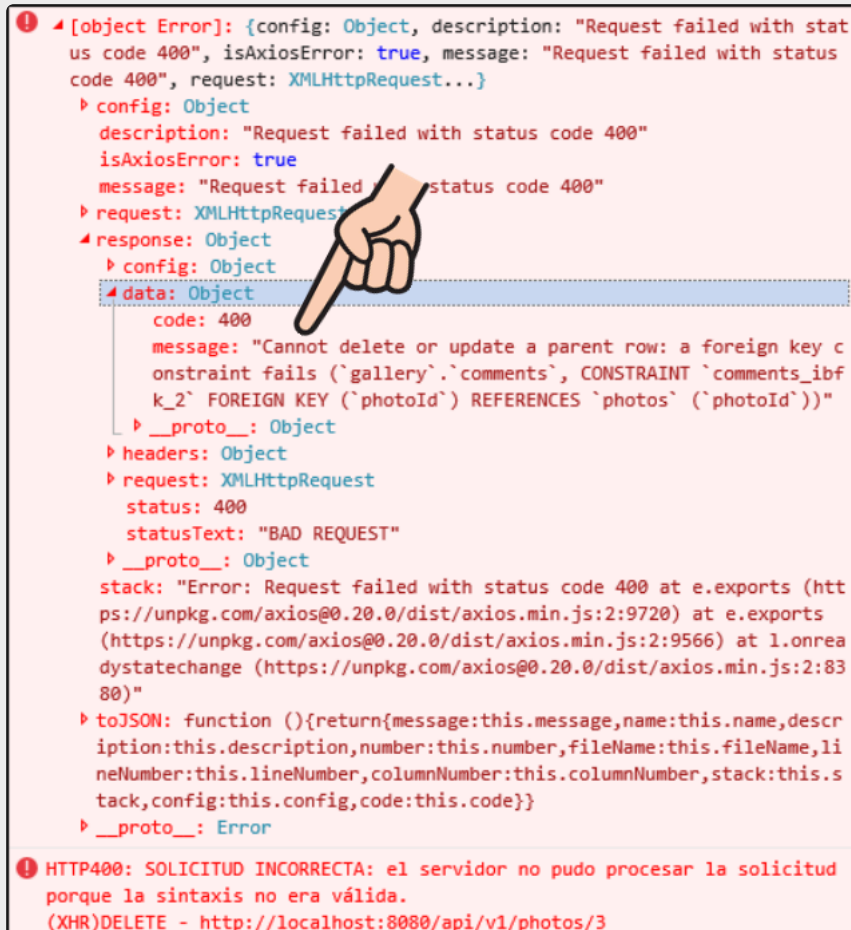
```
◀ [object Object]: {lastId: 0}  
  lastId: 0  
  ▶ __proto__: Object
```

Petición DELETE (II)

Al igual que el resto de métodos, si el servidor devuelve un error, se ejecuta el `catch`.

Foto con comentarios:

```
await axios.delete("/api/v1/photos/3")
```



```
[Object Error]: {config: Object, description: "Request failed with status code 400", isAxiosError: true, message: "Request failed with status code 400", request: XMLHttpRequest...}
  config: Object
    description: "Request failed with status code 400"
    isAxiosError: true
    message: "Request failed with status code 400"
  request: XMLHttpRequest
  response: Object
    config: Object
    data: Object
      code: 400
      message: "Cannot delete or update a parent row: a foreign key constraint fails ('gallery`.`comments`, CONSTRAINT `comments_ibfk_2` FOREIGN KEY (`photoId`) REFERENCES `photos` (`photoId`))"
      __proto__: Object
    headers: Object
    request: XMLHttpRequest
      status: 400
      statusText: "BAD REQUEST"
    __proto__: Object
  stack: "Error: Request failed with status code 400 at e.exports (https://unpkg.com/axios@0.20.0/dist/axios.min.js:2:9720) at e.exports (https://unpkg.com/axios@0.20.0/dist/axios.min.js:2:9566) at l.onreadystatechange (https://unpkg.com/axios@0.20.0/dist/axios.min.js:2:8380)"
  toJSON: function () {return {message: this.message, name: this.name, description: this.description, number: this.number, fileName: this.fileName, lineNumber: this.lineNumber, columnNumber: this.columnNumber, stack: this.stack, config: this.config, code: this.code}}
  __proto__: Error

! HTTP400: SOLICITUD INCORRECTA: el servidor no pudo procesar la solicitud porque la sintaxis no era válida.
(XHR)DELETE - http://localhost:8080/api/v1/photos/3
```

Contenidos

REST

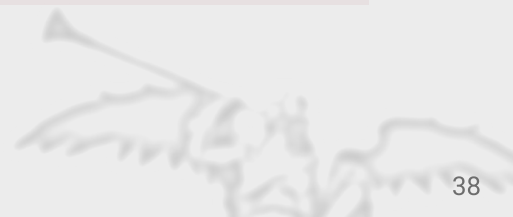
Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

Ejemplos

Módulos de comunicación con API



Módulos API

Módulos JS encargados de encapsular las peticiones a una API REST externa

Permiten aislar al resto de la aplicación de la implementación concreta para realizar peticiones AJAX proporcionando una interfaz común estándar

Acceso programático a las consultas, modificaciones, inserciones y borrado de datos

Centralizan la configuración de elementos comunes como la URL base o las cabeceras que deben enviarse en todas las peticiones (por ejemplo, tokens de sesión)



Estructura del proyecto

Organización del código

♦ Nosotros seguiremos la siguiente estructura



Módulo de configuración común de API

/js/api/common.js :

```
"use strict";  
  
import { sessionManager } from "/js/utils/session.js";  
  
const BASE_URL = "/api/v1";  
  
const requestOptions = {  
  headers: { Token: sessionManager.getToken() },  
};  
  
export { BASE_URL, requestOptions };
```

Se define la URL base y la configuración común para todas las peticiones

Módulo de acceso a API para la entidad Fotos

js/api/photos.js :

```
import { BASE_URL, requestOptions } from '../common.js';

const photosAPI = {
  getAll: async function() {
    let response = await axios.get(`${BASE_URL}/photos`, requestOptions);
    return response.data;
  },
  create: async function(formData) {
    let response = await axios.post(`${BASE_URL}/photos`, formData,
    requestOptions);
  },
  update: async function(formData, photoId) {
    let response = await axios.put(`${BASE_URL}/photos/${photoId}`, formData,
    requestOptions);
  }
};

export { photosAPI };
```

- Se define un objeto de acceso a API por cada entidad, con un método por cada operación
- Cada operación se define con una función asíncrona
- En el interior de la función se realiza la petición AJAX con axios
- Se usa la URL base y las opciones comunes
- Cuando el servidor responda, se devuelven los datos obtenidos de la API
- Los métodos pueden aceptar parámetros si los necesitan, por ejemplo, datos de formulario para enviar

Recordatorio

Los módulos de API se pueden generar automáticamente con el comando:

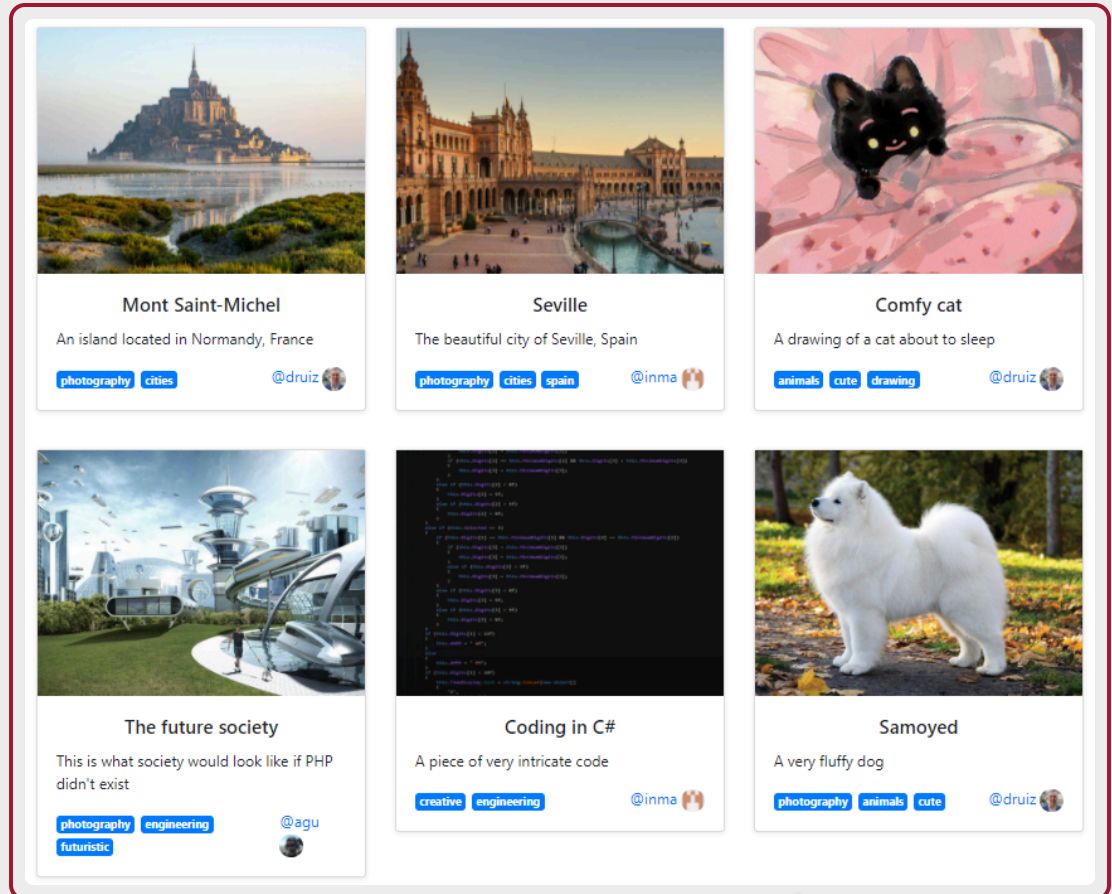
```
silence createapi
```

Ejemplo de uso: API + renderizadores

js/index.js :

```
import { photosAPI } from '/js/api/photos.js';
import { galleryRenderer }
  from '/js/renderers/gallery.js';
import { messageRenderer }
  from '/js/renderers/messages.js';

async function loadPhotos() {
  try {
    let galleryContainer =
      document.getElementById("gallery-container");
    let photos = await photosAPI.getAll();
    let cardGallery =
      galleryRenderer.asCardGallery(photos);
    galleryContainer.appendChild(cardGallery);
  } catch (err) {
    messageRenderer.
      showErrorAlert("Error showing photos", err);
  }
}
```



Conclusiones

Las aplicaciones web modernas intercambian datos con el servidor usando **APIs REST** y formatos ligeros como **JSON**.

Las peticiones **asíncronas** permiten mantener la interfaz reactiva mientras se consulta o actualiza información remota.

Librerías como **axios** simplifican el uso de `GET`, `POST`, `PUT` y `DELETE`, así como el tratamiento de respuestas y errores.

Organizar el acceso al servidor en **módulos de API** facilita reutilizar código, separar responsabilidades y conectar datos con la interfaz mediante renderizadores.



Gracias

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

UNIVERSIDAD DE SEVILLA